

MAXX STEELE™

PROGRAMMER'S TECHNICAL REFERENCE GUIDE



RoboForce

Version 1.0 | May 24, 2024

Maxx Steele Technical Manual

Maxx-Steele-1984-Robot contributors

2026-06-27

Contents

Maxx Steele Technical Manual	4
What this guide covers	4
How to use this reference guide	5
Applications guide	5
Firmware images in this archive	5
Chapter 1 — Bytecode Programming Rules	7
Introduction	7
Program storage	7
Operating modes (\$0D)	7
Status bytes	7
Program pointers (zero page)	8
Programming techniques	8
Beginner checklist	8
Chapter 2 — Opcode Vocabulary	9
Motion and I/O opcodes (\$00-\$10)	9
Extended opcodes (\$80-\$FF)	9
Keypad-mapped opcodes	10
Alphabetical index	10
Unknown opcodes	10
Chapter 3 — Programming Motion and Display	11
Motion overview	11
Drive opcodes	11
Arm and wrist opcodes	11
Home opcode \$0B	11
Lamp opcode \$0A	11
LED segment display	12
Programming techniques	12
Known gaps	12
Chapter 4 — Programming Speech and Music	13
Speech overview	13
ROM phrases (\$82 / \$0E)	13
RAM phrases (\$83)	13
Music overview	14
Song number opcode \$0D	14

Advanced notes	14
Chapter 5 — Internal ROM Operating System	15
What the internal ROM is	15
Boot sequence	15
Main loop and mode dispatch	16
Mode-specific behavior	17
Bytecode executor	19
Device drivers	20
Cartridge bootstrap	21
6502 vs bytecode — when to use which	23
Tools and simulation	24
Known gaps	24
Chapter 6 — Input/Output Guide	25
Memory map summary	25
Memory-mapped I/O (MMIO)	25
Remote keypad (RF input)	27
Cartridge expansion	31
On-board speech and face hardware	31
CPU and support chips	31
Schematics	31
Chapter 7 — UltraMaxx BASIC Language	32
Introduction	32
Getting started	32
Program format	32
Program size limits	33
UltraMaxx BASIC statements	33
Statement summary table	36
Copyright strings	36
Phrase and music tables	36
Sample programs	36
Compiler errors	37
What UltraMaxx BASIC is not	37
Relationship to other chapters	38
Quick command reference	38
Appendices	39
A — Opcode abbreviations	39
B — Display segment table	39
C — Memory map	39
D — Zero-page registers	39
E — Status bits	41
F — Music duration table	41
G — Cartridge file layout	41
H — tools/maxx_rom.py commands	42
I — Internal ROM entry points	42
J — Built-in speech phrases (game subset)	44
K — Messages and errors	44
L — Bibliography	45

M — MMIO hardware cross-reference	45
N — Glossary	45
Maxx Steele Quick Reference Card	46
Modes (\$0D)	46
Key zero page	46
Motion opcodes	46
Speech / music	46
Cartridge header (\$A000)	47
Common sequences	47
UltraMaxx BASIC (Ch 7)	47
Tools	47
Schematic Diagram Index	48
Main logic board	48
Transmitter (remote)	48
Receiver (robot RF)	48
Cartridge	48
Face / speech / display	48
Power module	49
Paddle mirror accessory	49
Chip cross-reference	49

Maxx Steele Technical Manual

This manual describes how to program the 1984 CBS Toys / Ideal **Maxx Steele** robot at the software level: bytecode motion programs, speech, music, cartridge images, and the 6502 internal ROM that interprets them.

It is modeled on the structure of the Commodore 64 Programmer's Reference Guide — numbered chapters, appendices, a quick-reference card, and a schematic index — adapted to Maxx Steele hardware.

PDF (latest): [Maxx-Steele-Technical-Manual.pdf](#) — front/rear covers plus chapters; re-built when `.md` or cover images (`cover-front.jpg`, `cover-rear.jpg`) change (`tools/build_technical` locally; GitHub Actions on push to main).

What this guide covers

Document	Audience	Focus
This guide	Programmers, reverse engineers	Bytecode language, memory map, I/O, internal ROM hooks
Cartridge/PROGRAMMING.md	Cartridge authors	Step-by-step cart layout, tools, demo walkthrough
Docs/User/	Owners	Setup, operation, games (not ROM layout)
Chassis/Manual/	Owners	Archival factory PDF scans
Docs/Mechanical/	Repairers	Chassis disassembly, reassembly, teardown photos

Primary sources: R. Wind disassemblies (maxx_internal_ROM.dsm, maxx_demo_ROM_532.dsm), tools/maxx_rom.py.

How to use this reference guide

If you are new to Maxx programming, read in order:

1. Chapter 1 — Bytecode programming rules
2. Chapter 2 — Opcode vocabulary
3. Chapter 3 — Motion and display
4. Work through the demo program in Cartridge/PROGRAMMING.md §5

If you are writing cartridges, add:

5. Chapter 5 — Internal ROM operating system
6. Chapter 7 — UltraMaxx BASIC language
7. Cartridge/PROGRAMMING.md §9 (authoring workflow)

If you are reverse engineering hardware, use:

- Chapter 6 — Input/output guide
- Schematics
- Annotated .dsm listings under Mainboard/Firmware/ and Cartridge/Examples/

Keep Quick reference and Appendices open while coding.

Applications guide

Application	Mode (\$0D)	What you need
Remote control (immediate)	0	Remote keypad; opcodes executed live
Learn key sequences	1	Records into program RAM
Enter program steps	2	Keypad maps to opcodes via \$E6B5 table
Run stored program	3	Bytecode at \$0200, terminator FF FF
Built-in games (Moon Ball / Force Field)	4	JMP (\$0094) → \$F8CE; \$F8E9 / \$FAA4 — see Ch 5 + User Manual Ch 6
Factory demo	3 + cart	CBSDemo.532 bootstrap
Custom cartridge	3 + cart	4 KB EPROM image; see Ch 5-7 + cartridge manual
UltraMaxx BASIC authoring	3 + cart	Ch 7; tools/maxx compile → .532
Speech / music authoring	any	Ch 4; phrase tables in cart or RAM
Repair / RE	—	Ch 6, Schematics, DataSheets/

Firmware images in this archive

Image	Path
Internal 8 KB ROM	Mainboard/Firmware/Binary/Maxxrom.64
Demo cartridge (4 KB)	Cartridge/Examples/CBSDemo/Firmware/Binary/
UltraMaxx cartridge (4 KB)	Cartridge/Examples/UltraMaxx/Firmware/Binary/

Tools: `tools/maxx` — compile UltraMaxx BASIC, validate, list, upload; `tools/maxx_rom.py` — disassemble, template, opcode export.

Chapter 1 — Bytecode Programming Rules

This chapter describes the rules for writing Maxx Steele **programs**: how bytecode is stored, how operating modes affect entry, and practical techniques for building sequences.

The robot does not run arbitrary user 6502 code for motion demos. The internal ROM interprets a **bytecode program** in RAM. Cartridges supply tables and a short bootstrap stub; see Chapter 5.

Introduction

A Maxx program is a linear sequence of (**opcode, operand**) byte pairs. Each pair is one step. The executor reads from program RAM, displays the opcode name on the LED segment display, and performs the action.

Programs are edited through the remote keypad (Program mode), loaded from a cartridge bootstrap, or built offline and burned into EPROM.

Program storage

Property	Value
RAM region	\$0200-03FE
Step size	2 bytes (opcode, operand)
Maximum steps	255 pairs (512 bytes); FULL displayed when \$037F exceeded
Terminator	FF FF (required)

Example minimal program:

```
83 10    ; speak RAM phrase 0
0C 03    ; delay 3 seconds
01 10    ; drive forward
0B 00    ; home all joints
FF FF    ; end
```

Operating modes (\$0D)

The mode byte in zero page selects how keypad input and the executor behave.

Value	Name	Behavior
\$00	Immediate	Keys execute motion/speech instantly
\$01	Learn	Records key sequences into program RAM
\$02	Program	Enter bytecode steps via keypad
\$03	Execute / Game	Run the stored program

Cartridge entry stubs typically initialize status bytes \$02 and \$03 before jumping to the internal main loop at \$E0B6.

Status bytes

\$02 — Status A

Set by cartridge bootstrap (demo uses LDA #\$02 / STA \$02). Bits include speech error (**Ser**), enable backoff (**Ebof**), power-down, drive, and speech flags. Keypad rows toggle subsets; see Appendix E.

\$03 — Status B

Demo entry uses LDA #\$82 / STA \$03 (**PDon**, **Edof**, **SPof**, **UCon**). Governs user control and execute gating.

Program pointers (zero page)

Addr	Role
\$0F / \$10	Current step in \$0200 table (program mode / execute)
\$11 / \$13	Current opcode and operand during entry
\$24 / \$25	Program byte pointer used by executor

Programming techniques

Delays

Opcode \$0C operand is delay time in **seconds** (decimal). Chain delays between motion commands so mechanics can finish.

0C 02 ; wait 2 seconds

01 14 ; forward (operand scale empirical – see Ch 3)

0C 04 ; wait 4 seconds

Homing

Opcode \$0B with operand **\$00 only** — returns joints to a known position before the next move.

Speech then motion

RAM phrases (\$83) must be copied to \$0500 before execute (cartridge stub or prior setup). ROM phrases (\$82) need no RAM table.

Program length

If program entry exceeds available RAM, the display shows **FULL** (internal ROM compares against \$037F).

Marker opcode \$FE

Display-only **beg** marker; does not affect execution flow in the demo cart.

Beginner checklist

1. Every program ends with FF FF.
2. Operands are one byte (\$00-FF); meaning depends on opcode (Ch 2-4).
3. Test with python3 tools/maxx_rom.py validate (see tools/maxx_rom.py) before burning EPROM.
4. For a worked 38-step demo, see Cartridge/PROGRAMMING.md §5.

Next: Chapter 2 — Opcode vocabulary

Chapter 2 — Opcode Vocabulary

This chapter lists every known Maxx Steele program opcode. Display names come from the internal ROM segment table at **\$F878** (see Appendix B).

Opcodes are described in **hex order** within each group. For a one-page summary, see Quick reference.

Export machine-readable opcode JSON (tools/maxx_rom.py; output path is arbitrary, e.g. under Cartridge/):

```
python3 tools/maxx_rom.py opcodes Cartridge/opcodes.json
```

Motion and I/O opcodes (\$00-\$10)

Opcode	Display	Name	Operand
\$00	L	Turn / drive left	Distance or angle
\$01	F	Drive forward	Distance
\$02	b	Drive reverse	Distance
\$03	r	Turn / drive right	Angle
\$04	Uu	Wrist up	Value
\$05	Ud	Wrist down	Value
\$06	Au	Arms up	Value
\$07	Ad	Arms down	Value
\$08	Cr	Claw rotate	Value
\$09	Cc	Claw open/close	\$00=open, \$01=close
\$0A	HL	Lamp	\$00=off, \$01=on
\$0B	init	Home (all joints)	Must be \$00
\$0C	d	Delay	Seconds
\$0D	Sn	Song number reference	Index
\$0E	S	Speech (ROM phrase)	Phrase #
\$0F	SS	Speech (shift mode)	Phrase #
\$10	PS	Program speech capture	Slot #

Examples

```
01 14    ; forward
0A 01    ; lamp on
0C 0A    ; delay 10 seconds
82 3F    ; speak ROM phrase $3F
```

Extended opcodes (\$80-\$FF)

Opcodes with bit 7 set (\$80+) are remapped through the display table for execution. The executor treats \$80 as an alias mapping to display index \$0C.

Opcode	Display	Name	Operand
\$81	PLAY	Play tune from music RAM	Tune # (\$0400 table)
\$82	SPEE	Speak ROM phrase	Phrase #
\$83	SS	Speak RAM phrase	Phrase # in \$0500
\$84	CLr	Clear speech phrase slot	Slot #
\$FE	beg	Program begin marker	Display only
\$FF	End	End of program	Must be \$FF (pair FF FF)

Keypad-mapped opcodes

The internal ROM maps remote key scan codes to opcodes via table **\$E6B5**. Faceplate names: Transmitter/remote-keypad.md. Documented mapping (from maxx_internal_ROM.dsm):

Key opcodes: 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F
41 46 43 13 84 82 81 83 80

Opcode	Typical keypad use
\$41	Game-related step
\$43	Click
\$46	Motion step
\$13	(mapped key — see learn/program mode)
\$80-\$84	Extended speech/music/clear group

In **Program mode**, pressing a key stores the mapped opcode and prompts for an operand byte.

Alphabetical index

Name	Opcode
arms_down	\$07
arms_up	\$06
claw_open_close	\$09
claw_rotate	\$08
delay	\$0C
drive_forward	\$01
drive_reverse	\$02
end	\$FF
home	\$0B
lamp	\$0A
play_tune	\$81
song_number	\$0D
speak_ram	\$83
speak_rom	\$82
speech_clear	\$84
speech_program	\$10
speech_rom	\$0E
speech_shift	\$0F
turn_left	\$00
turn_right	\$03
wrist_down	\$05
wrist_up	\$04

Unknown opcodes

If the executor encounters an undefined opcode, behavior is undefined in this documentation. Stick to opcodes listed above and validated with tools/maxx_rom.py.

Previous: Chapter 1 · **Next:** Chapter 3 — Motion and display

Chapter 3 — Programming Motion and Display

This chapter describes motion opcodes, the headlamp, and how the LED display shows program steps during entry and execution.

Distance and angle **operands are empirical** — the internal ROM scales them to motor timing. Values in the factory demo (e.g. \$14 forward, \$28 arm raise) are starting points, not documented engineering units.

Motion overview

Drive and joint motion opcodes \$00-\$09 write timing values to motor control hardware mapped around **\$1600** and **\$1C00** (see Chapter 6).

The executor runs each step to completion (or until interrupted) before advancing the program pointer at \$0200.

Drive opcodes

Opcode	Action	Demo cart examples
\$00	Turn left	00 05
\$01	Forward	01 14, 01 10
\$02	Reverse	02 14
\$03	Turn right	03 06

Operands control how long or how far the drive system runs. Compare steps in `maxx_demo_ROM_532.dsm` with on-robot behavior when calibrating new values.

Arm and wrist opcodes

Opcode	Action	Demo example
\$04	Wrist up	—
\$05	Wrist down	05 23
\$06	Arms up	06 28
\$07	Arms down	—
\$08	Claw rotate	08 15
\$09	Claw open/close	09 00 (open)

Home opcode \$0B

Always use operand **00**:

0B 00 ; home – all joints to reference position

The demo cart homes before backing up at the end of the sequence.

Lamp opcode \$0A

Operand	Effect
\$00	Lamp off
\$01	Lamp on

Demo sequence: 0A 01 ... delay ... 0A 00.

LED segment display

During program entry and execution, the internal ROM fetches a **two-byte display code** from table **\$F878** indexed by the current opcode (see LDA \$F878,Y in maxx_internal_ROM.dsm). This drives the shift-register display at **\$1200**.

Display names in Chapter 2 match that table (e.g. **L**, **F**, **PLAY**, **d**).

When program RAM is full, the display spells **FULL** (bytes at internal ROM label referencing \$037F limit).

Programming techniques

1. **Home after motion blocks** — reduces accumulated joint error.
2. **Delay after every drive** — allows the chassis to settle before the next command.
3. **Lamp as status** — signal “busy” during long speech delays.
4. **Small operand sweeps** — test \$01 08, \$01 10, \$01 14 on your robot and record what works.

Known gaps

- Exact mm/degree mapping for operands is not documented in the internal ROM listing.
- Motor ramp profiles and limit switches are hardware-dependent; see Mainboard/Schematic/ and Mainboard/KiCAD/ (digitization in progress).

Previous: Chapter 2 · **Next:** Chapter 4 — Speech and music

Chapter 4 — Programming Speech and Music

This chapter describes how the Maxx Steele robot produces speech and music under program control.

Speech uses a parallel phoneme interface at **\$1400**. The speech driver subroutine **\$F3D5** clocks 4-bit nybbles to the synthesizer. Music uses byte pairs in RAM at **\$0400** and an IRQ handler near **\$F1D8**.

Speech overview

Mechanism	Opcode	Data source
ROM phrase table	\$82, \$0E	Internal ROM phoneme strings
RAM phrase table	\$83	\$0500 (copied from cartridge or recorded)
Program speech capture	\$10	User-recorded slots
Clear phrase slot	\$84	Clears RAM slot

ROM phrases (\$82 / \$0E)

Phrases are indexed phoneme strings in internal ROM. The driver at **\$F3D5** outputs sounds with index **X** (see JSR **\$F3D5** in `maxx_internal_ROM.dsm`).

Phoneme code tables reside at **\$F567 / \$F4DB** (140 entries). Until the full **\$F4DB** table is transcribed, treat phrase indices as opaque IDs.

Demo examples:

```
82 3F      ; "Ha ha ha ha ha" (ROM)
83 16      ; uses ROM index via RAM phrase indirection in demo – see cart listing
```

RAM phrases (\$83)

Cartridges embed phrase data at offset **\$81** (address **\$A081** for demo cart). The bootstrap copies bytes to **\$0500**.

Each phrase is a sequence of **16-bit phoneme tokens**; **\$FF** pads unused slots.

Demo cart phrase table (from `maxx_demo_ROM_532.dsm`):

Slot	Content (abbrev.)
0	“Hello, I am Maxx Steele” / greeting tokens
1	“I am ready when you are”
2	“I am a great match for humans”
3	“Goodbye for now, have a good day”

Example program bytes:

```
83 10      ; speak RAM phrase 0 (demo uses operand $10 for table indexing –
verify in .dsm)
0C 02      ; delay
83 00      ; another RAM phrase
```

Note: Demo listing uses operands \$10, \$00, \$01, etc. — cross-check against your cart image with `tools/maxx_rom.py disasm`.

Music overview

Opcode **\$81** plays a tune from the music RAM table at **\$0400**.

Tune selection calls **\$EF01** to resolve pointers for tune number in the operand (multiple call sites in internal ROM).

Music byte pairs

Stored in **\$0400** (and cartridge offset **\$BB / \$A0BB**):

```
70 12    ; note
70 11    ; note
38 0F
54 0D
E0 0F
00 00    ; end of tune
```

The music IRQ handler at **\$F1D8** reads duration data from **\$F15B** and frequency/note data from **\$0400**.

Demo: 81 06 plays tune #6 (Reveille); 81 00 plays tune #0 at end of demo sequence.

Song number opcode \$0D

References a song index for the internal song table (distinct from \$81 play-tune in some keypad contexts). Used in learn/program flows.

Advanced notes

- **Phoneme authoring** requires transcribing **\$F4DB / \$F567** — listed as future work in Cartridge/PROGRAMMING.md §10.
- **Speech error flag (SEr** in **\$02**) — set when speech subsystem reports fault; demo cart entry sets related status.
- Face/speech hardware datasheets: DataSheets/ (Sanyo LC3100/LC8100, ET9420 family).

Previous: Chapter 3 · **Next:** Chapter 5 — Internal ROM OS

Chapter 5 — Internal ROM Operating System

This chapter documents the robot's **8 KB masked ROM** (\$E000-\$FFFF): boot, operating modes, the main interpreter loop, device drivers, and how cartridge bootstrap stubs hand off to it.

Cartridge layout and authoring workflow remain in § Cartridge bootstrap below and in Cartridge/PROGRAMMING.md.

Primary sources: Mainboard/Firmware/Assembly/maxx_internal_ROM.dsm (R. Wind), Mainboard/Firmware/Binary/Maxxrom.64, tools/maxxbas/patches.json (simulator traps).

What the internal ROM is

At power-on the **MOS 6502** executes firmware in **\$E000-FFFF**. This is the robot's only native execution environment — analogous to a C64 KERNAL, but specialized for:

- Cold/warm boot and RAM initialization
- Cartridge detection and table loading
- Keypad scan and **five operating modes** (\$0D = 0-4)
- Bytecode fetch/decode from \$0200
- Speech, music, motor, and display drivers
- Built-in games (mode 4)

User motion programs are **not** generally written in 6502. They are opcode/operand pairs in RAM, interpreted by this ROM. Cartridges supply a short **6502 bootstrap** plus data tables; they do not replace the interpreter.

Boot sequence

Reset and copyright check

Address	Role
\$E041	RESET entry — SEI, init \$1E00 and display, set stack
\$E04F-\$E058	Compare ROM copyright at \$E000 with RAM mirror \$0100
\$E05A	Cold start — zero page, phrase RAM \$0500-\$05FF ← \$FF, clear music
\$E06F	Warm start — refresh copyright mirror, copy vector table

Cold start calls:

1. \$EF55 — zero music table \$0400-\$04FF
2. \$E582 — init program pointers and flags
3. \$E57B — set program terminator at \$0180 ← \$FF

Warm start copies the **37-byte vector table** from ROM \$E01C into RAM **\$72-\$96**, then calls \$EEEE (timer init).

RAM vector table (\$72-\$96)

Copied from ROM at boot. Key entries (initial values from listing):

ZP	Points to	Purpose
\$76	\$E014	NMI handler
\$78	\$FDC8	IRQ handler
\$7A	\$E0CB	Immediate-mode return (main loop)

ZP	Points to	Purpose
\$7C-\$85	\$F1D8 region	Music IRQ jump table
\$8C	\$EE32	Opcode dispatch (JMP (\$008C) at \$EE2F)
\$8E/\$8F	\$1000	Cartridge scan pointer (page/bank)
\$90	\$E598	Alternate dispatch
\$92	\$F3D8	Speech output vector
\$94	\$F8CE	Game mode entry

Full table: Appendix D — Zero-page registers.

Cartridge scan

After vectors are loaded, the ROM walks 4 KB boundaries from **\$2000** through **\$B000**:

1. Read 16-bit entry vector at cart base (\$8E/\$8F)
2. Compare bytes at offset +2 against ROM copyright template ((c) 1985 CBS Toys)
3. On match: store vector in \$8E/\$8F and **JMP (\$008E)** into cartridge bootstrap
4. On no match: fall through to **\$E0B6** (main loop without cart)

The factory demo cart at **\$A000** is the reference bootstrap; see Cartridge bootstrap.

Main loop and mode dispatch

Entry: **\$E0B6**

Whether or not a cartridge was found, execution eventually reaches **\$E0B6**:

1. Display “9” on the LED (\$ED4F / \$ED7B)
2. Greet: “Hello, I’m Maxx Steele.” (\$F475, phrase \$10) if speech enabled
3. Check stalled motors (\$EDAF)
4. Set **\$75 ← \$80** (keypad ready), **\$0D ← 0** (immediate mode)
5. Load startup tune 1 (\$EF01)
6. Call \$E905 (status/key housekeeping)
7. Dispatch on **\$0D** at **\$E0ED**

Mode variable **\$0D**

Value	Mode	Display label	Entry
0	Immediate	—	\$E154 — live key execution
1	Learn	LErn	\$E161 — record key sequences
2	Program	Prog	\$E346 — enter bytecode via keypad
3	Execute	run	\$E434 — run program at \$0200
4	Game	PLAY	JMP (\$0094) → \$F8CE

Modes 1–4 show a two-character LED label (\$0D + 5 → index into display strings at \$F684). Mode changes re-enter through **\$E0D1** (STA \$0D then fall into the dispatch chain). The **GAME** remote key and clock/status paths also jump to \$E0D1 with the new mode in A.

Shared busy flag: **\$2B**

Many paths call **\$EF63** first — a tight loop until **\$2B = 0**. The listing marks \$2B as a motor/speech **talkback** busy flag (motor controller confirms motion before the next command). The simulator bypasses this wait at \$EF63 (patches.json).

Keypad input: **\$75 / \$15**

Decoded remote keys land in **\$75** (bit 7 set when a key is pending). **\$E617** polls the RF/matrix path; **\$E6A4** copies the keycode to **\$15** and clears the ready bit in \$75.

Trap addresses for debugging: \$E161 (learn loop), \$E17B (immediate decode), \$E617 (poll), \$EE2F (opcode dispatch).

Mode-specific behavior

Immediate (mode 0) — \$E154 / \$E161 / \$E17B

- Says “*I’m ready.*” (phrase \$20) when speech is on
- **\$E161** loop: \$E617 → test \$15
- **\$E17B**: key in A — WAIT (\$0C), motion keys, PLAY, SPEECH, etc.
- Motion keys call **\$EF2E** (drive) or **\$EF40** (stop); PLAY resolves tune via **\$EF01**
- **\$E6B5** table maps keycodes 0–\$13 to opcode bytes for learn/program paths

Learn (mode 1) — \$E161 / \$E574

- Init program pointer (\$E574 sets \$0200 terminator, \$0F/\$10 ← \$0180)
- Prompt: “*Please teach me.*” (phrase \$1E)
- Records key sequences into \$0200; CLEAR (\$0E) calls **\$E3A9**
- Shift/CLEAR keys move or prepend steps (\$E2A2 region)

Program (mode 2) — \$E346

- Shows current step number (\$F64E)
- Keypad maps through **\$E519** → opcode in **\$11**, operand in **\$13**
- **\$E3F9** / **\$E409** — step entry and RUN (ENTER) preview via **\$F66C**
- OUT OF SPACE → “*Sorry, my circuits are full*” (\$F40F)
- BEGIN (\$13) decrements pointer; END markers \$FF/\$FE

Execute (mode 3) — \$E434

- Sets run flag, walks (**\$0F**, **\$10**) through \$0200
- Each step: **\$E519** → **\$F66C** (execute one command) → **\$E600** (wait for \$75)
- **\$FF** ends program; returns to immediate via **JMP (\$007A)**
- Low battery: **\$96** counter triggers “*I need energy, please recharge me.*” during execute/game

Game (mode 4) — \$F8CE

Player-facing rules for the two built-in games are in User Manual Ch 6 — Games (*Moon Ball* and *Force Field*). This section maps that behavior to ROM entry points and zero-page state.

Mode entry: **\$94** → **\$F8CE**. The **GAME** remote key sets **\$0D** ← **4** via **\$E0D1** (not mode 3 — execute is mode 3). Display label **PLAY** comes from the mode string table at **\$F684** (\$0D + 5 → index).

Shared entry flow (\$F8CE–\$F8E6)

Step	Address	Action
1	\$EBDC	HOME arms/wrist/claw (game preamble)
2	\$EF63	Wait for motor talkback (\$2B = 0)
3	\$F9EA	Say phrase \$12 — “ <i>Please choose game.</i> ”
4	\$E9FE	Wait for digit + ENTER ; operand \$20 in \$13 , selector \$05 in A
5	\$F8E4	Operand 0 → Moon Ball (\$F8E9); non-zero → Force Field (\$FAA4)

Both games call **\$F9DE** (zero **\$A6–\$AD**) and **\$F9E8** (difficulty picker: phrase **\$11**, accept **0–3**,

return index in X).

Moon Ball — game 1 (\$F8E9) Reflex game: reflect the headlamp into the photo transistor between the eyes when the lamp flashes.

User Manual rule	ROM implementation
Difficulty 0-3	\$F9E8 → \$A6
Ready: reflect light within ~20 s	\$27 ← \$14 IRQ countdown; BIT \$1200 / BVS at \$F91D-\$F920
Say “I’m ready.”	Phrase \$20 at \$F90C
3 misses → game over	\$AC ← 3 at \$F8FB; DEC \$AC on timeout at \$F98F
“Good play” + score on display	Phrase \$13 at \$F99C; BCD add \$A6 → \$A7/\$A8 ; \$FA01 copies to \$24/\$25
Difficulty ramps (shorter window, dodge, more points)	Tables \$FA90-\$FAA0 : seconds (\$FA90), dodge delay (\$FA94), points per return (\$FAA0); \$A6 incremented after \$A9 rounds expire
Game over + replay	\$F9C6 : phrase \$1B ; \$FA67 : phrase \$1C , wait for ENTER (\$27 ← \$10); timeout → immediate via JMP (\$007A)

Main play loop: **\$F92C** (random drive + flash) → **\$F996** (hit) or miss path → **\$F9C6** when **\$AC** underflows.

Photo input uses the display/MMIO path at **\$1200** (see Chapter 6); the faceplate photo transistor sits in the light-return path described in the User Manual.

Force Field — game 2 (\$FAA4) Two-player-style duel: player shield/laser on the remote vs. Maxx shield/laser (siren + warble + headlamp).

User Manual rule	ROM implementation
Difficulty 0-3	\$F9E8 → \$A6 ← X × 4 at \$FAAD-\$FAAF
Shield = hold <0>, laser = hold <3>	\$FA0C reads keypad; mask table \$FA98 ; \$FB8C shield/laser state machine
Scores: player left, Maxx right on display	BCD \$A7 (player) / \$A8 (Maxx) → \$24/\$25 at \$FC34-\$FC3C
1-in-4 ricochet off Maxx shield	\$FBB1/\$FBB5 : compare \$66/\$69 to 1 or 4 ; penalty via \$FC12 (subtract from \$A7)
First to 25 points wins	CMP #25 at \$FC30
~20 s levels + intermission tune	Level timer \$28 (scaled by \$A6 at \$FB4B); \$FBCE intermission; \$2A stun countdown
Win / lose speech	Phrase \$18 (<i>Maxx Steele wins</i>) or \$19 (<i>Congratulations, you win</i>) at \$FC4A-\$FC50
Replay	\$FA67 after win sequence at \$FC69

IRQ-driven timing for level ticks lives in **\$FB8C-\$FC6C** and the game slice of the IRQ handler at **\$FC6F / \$FDCD**.

Game state variables (\$A6-\$AD)

ZP	Moon Ball	Force Field
\$A6	Difficulty index; also points added per successful return (ramps to 3)	Difficulty × 4; level index incremented between rounds
\$A7	Player score (BCD low)	Player score (display left / \$24)
\$A8	Player score (BCD high)	Maxx score (display right / \$25)
\$A9	Rounds-until-ramp counter (starts 3)	Phase/level sub-counter
\$AA	—	Sub-phase timer (shield down, laser burst)
\$AB	—	Keypad-derived shield/laser flags
\$AC	Misses remaining (starts 3)	—
\$AD	Cleared by \$F9DE ; role TBD	Cleared by \$F9DE ; role TBD

Full ZP catalog: Appendix D — Zero-page registers.

Game speech phrases (ROM index ↔ factory manual #) Factory manual lists phrases **16-32**; ROM stores them at **\$F640** with indices **\$10-\$20**. Game-related entries:

Manual #	ROM index	Text
17	\$12	Please choose game.
18	\$11	Please choose how tough.
19	\$13	Good play.
24	\$18	Maxx Steele wins.
25	\$19	Congratulations, you win.
27	\$1B	Game over.
28	\$1C	Choose enter to play again.
32	\$20	I'm ready.

Say path: **\$F40F** (index in X). Greeting uses **\$F475** for phrase **\$10** at main loop entry.

Bytecode executor

One step: \$F66C

Called from program mode preview, execute loop, and learn playback:

1. **\$F737** — compute 1-based step number from \$0F/\$10
2. Load **\$11** (opcode), **\$13** (operand)
3. **\$FF** → display **End**, exit; **\$FE** → display **beg**
4. **\$0B (HOME)** → four-byte init pattern from \$F7F8-\$F858 tables
5. Opcodes \$80-\$87 remap to table indices \$0C-\$13
6. Display two-character name from **\$F878** table; show **\$24/\$25** as hex for extended opcodes
7. **JMP (\$008C)** at **\$EE2F** — dispatch to motion/speech/music handlers

Addr	Role
Program pointer	
Addr	Role
\$0F/\$10	Current step address in \$0200 (starts \$0180 = byte offset into table)
\$11/\$13	Current opcode / operand
\$24/\$25	Executor working pointer (displayed for \$8x opcodes)

Increment: **\$E41A** (+2); decrement/BEGIN: **\$E555**.

Device drivers

Display (\$1200)

Address	Role
\$ED4F	Shift LO bit to display chain
\$ED7B	Send one byte to \$1200
\$ED48	Display prefix / spacing
\$F684	Send null-terminated 2-char strings (mode labels, tune names)
\$F878	Opcode → LED segment patterns (Appendix B)

Simulator bypasses serial wait loops at \$ED5F–\$EDA0 (patches.json).

Speech (\$1400)

Address	Role
\$F3D5	Speech output entry (JMP (\$0092) → \$F3D8)
\$F460 / \$F465	Clock nybbles to \$1400
\$F567 / \$F4DB	Phoneme/duration tables (140 entries, index in X)
\$F475	Say built-in ROM phrase by index (table at \$F640)
\$F40F	Say phrase with index in X (games, errors, prompts)

Built-in phrases \$10–\$20 include greetings, mode prompts, and game text (see listing at \$F640).

Address	Role
---------	------

Music

Address	Role
\$EF01	Resolve tune pointer for operand (tune table in ROM)
\$F15B	Note duration table (indexed 1-8)
\$F1D8	Music IRQ handler region
\$F0B8	Add note pair to \$0400
\$EF55	Clear music RAM

IRQ path decrements **\$26, \$28, \$2A, \$27** for note timing and game delays.

Motors (\$1600, \$1C00, \$1E00)

Address	Role
\$EF2E	Motor drive — Y = bank, A = command nibble
\$EF40	Stop all motors (Y=\$01, A=\$6F)
\$EF63	Wait for talkback (\$2B = 0)
\$EDAF	Stalled-motor check (main loop)
\$EF76	Leave learn mode / motor cleanup

MMIO bitfields and CPU pin mapping: Chapter 6, Mainboard/Schematic/MMIO-Pin-Map.md.

Cartridge bootstrap

What a cartridge is

A **4 KB EPROM** maps into **\$2000-\$B000** in 4 KB steps. The factory demo sits at **\$A000**.

A valid image supplies:

1. Entry vector and copyright header
2. A short **6502 bootstrap stub**
3. Program, speech, and music **data tables**

The stub copies tables into RAM and jumps to **\$E0B6** — it does **not** replace the interpreter.

ROM layout (\$A000 base)

Offset	Size	Content
\$00	2	Entry vector (word), e.g. \$A013
\$02	17	Copyright: (c) 1985 CBS Toys
\$13	code	Bootstrap stub (6502)
\$35	table	Program bytecode
\$81	table	Speech phrases (opcode \$83)
\$BB	table	Music notes (opcode \$81)

Cartridge detection compares the copyright string at offset +2 against the internal ROM template.

Demo stub sequence

Entry at **\$A013** (maxx_demo_ROM_532.dsm):

1. STA \$02 / STA \$03 — initialize status flags

2. Copy program from **\$A035 → \$0200**
3. Copy phrases from **\$A081 → \$0500**
4. Copy music from **\$A0BB → \$0400**
5. **JMP \$E0B6** — enter internal ROM main loop

Expected entry prologue: A9 02 85 02 (LDA #\$02 / STA \$02). tools/maxx_rom.py validate checks this.

Extending vs replacing the OS

Full OS replacement via cartridge alone is not possible. Reset always enters **\$E041** in the 8 KB mask ROM (\$E000-\$FFFF). That ROM stays mapped; there is no documented bank-out or cartridge overlay of the OS region. Replacing the entire firmware requires a new mask ROM, a piggyback/modified ROM chip, or similar hardware work—not the stock cartridge slot.

Partial extension is possible. Boot order matters:

1. Warm start copies the vector table from ROM **\$E01C → RAM \$72-\$96** (\$E085-\$E08B)
2. Cartridge scan: on copyright match, **JMP (\$008E)** into cart bootstrap (\$E0A9-\$E0B3)
3. Cart code runs **after** vectors are in RAM but **before** the main loop at **\$E0B6**

The factory stub only loads RAM tables and returns to **\$E0B6**. A custom bootstrap can do more in that window:

Technique	What it can change
RAM vector patch	Repoint entries in \$72-\$96 before handing off (see table below)
Extended cart 6502	Skip JMP \$E0B6 and run a cart-resident loop at \$A000-\$AFFF , calling ROM drivers via JSR
Data layer (factory pattern)	Replace program/phrases/music in RAM — bytecode interpreted by mask ROM

Reference example: Cartridge/Examples/Maxx0S/ — bootstrap at **\$A013** jumps to **\$A080** and runs a keypad math quiz using **\$E60D**, **\$F684**, **\$F475**, and **\$F8BE** display segments. No bytecode tables; tools/maxx_rom.py treats images without **JMP \$E0B6** as extended carts.

RAM vectors a cart bootstrap can repoint (defaults from ROM \$E01C):

ZP	Default target	Role
\$8C/\$8D	\$EE32	Opcode dispatch — JMP (\$008C) at \$EE2F
\$7A/\$7B	\$E0CB	Return to main loop (execute end, game replay timeout, ...)
\$94/\$95	\$F8CE	Game mode entry — JMP (\$0094)
\$92/\$93	\$F3D8	Speech output — JMP (\$0092)
\$78/\$79	\$FDC8	IRQ handler
\$76/\$77	\$E014	NMI handler

Cart handlers can **JSR** back into mask-ROM drivers (\$EF2E motors, \$F3D8 speech, \$ED7B display, ...) for hardware they do not reimplement.

Limits:

- Only a **few** OS paths use **JMP (\$00xx)** through these vectors; most firmware **JSR/JMPs fixed addresses** in \$E000-\$FFFF (mode dispatch, keypad poll, execute loop, IRQ internals).
- Cart space is **4 KB** per image; header and tables consume much of the factory layout.
- **Copyright required** for bootstrap to run; without a match, firmware goes straight to **\$E0B6**.
- Scan walks **\$2000-\$B000**; the **first** matching cart wins—no second bootstrap in the same boot.

Goal	Via cart bootstrap?	Notes
Custom motion/speech/music program	Yes	Bytecode + tables (factory pattern)
Custom game mode	Partially	Patch \$94 → cart code; mode entry/keypad still ROM
Custom opcode dispatch	Partially	Patch \$8C ; many paths bypass the vector
Replace keypad / mode / IRQ core	Practically no	Hardcoded ROM; IRQ timing critical
Replace entire OS	No	Mask ROM always owns reset and most code paths
Full custom firmware	Hardware	New mask ROM or piggyback; not cartridge alone

Patching **Maxxrom.64** itself (or simulator patches.json traps) is a separate path from cartridge bootstrap—see **6502 vs bytecode** below.

6502 vs bytecode — when to use which

Task	Language
Motion/speech demo sequence	Bytecode in \$0200
Custom phrases/music in cart	Data tables + bootstrap
Patch OS behavior (mask ROM or cart vectors)	6502 — see Extending vs replacing the OS above
New cartridge program	Bootstrap stub (minimal 6502) + bytecode tables
Debug firmware paths	maxx simulate + patches.json traps

Do not place arbitrary 6502 execution paths in the demo program area unless you fully understand cartridge mapping and internal ROM expectations.

Tools and simulation

Tool	Use
Cartridge/PROGRAMMING.md §9	Step-by-step cartridge authoring
tools/maxx_rom.py	template, validate, disasm
tools/maxx_simulate	Patched ROM + program trace + traps
Mainboard/Firmware/README.md	ROM binary and listing index

```
python3 tools/maxx_rom.py template mycart.532
python3 tools/maxx_rom.py validate mycart.532
maxx simulate Cartridge/Examples/CBSDemo/Firmware/Binary/CBSDemo.532 --cycles 30000
```

EPROM type: Mitsubishi KM2365 family — DataSheets/Mitsubishi-KM2365.pdf.

Known gaps

- **Game IRQ / timing tables** — decode \$FA90-\$FAA0 bytes to seconds and motor arcs; trace \$FB8C-\$FC6C and \$FDCD game tick to headlamp/motor \$EF2E commands
- **\$F9FE / \$E9FE** — keypad wait helper spans display-init bytes in the .dsm; re-anchor entry for game-select operand \$05
- **Full zero page** — only programmer-facing locations are catalogued (Appendix D)
- **I/O bitfields** — provisional tables in Ch 6 + MMIO pin map; glue IC refdes still TBD
- **Motor talkback** — \$EF63 / \$2B not traced to **MoCOP Done** / COP41xL pin yet
- **\$F222 motor serial** — target of \$EF2E/\$EF40; not fully disassembled in .dsm
- **Phoneme tables** — \$F4DB / \$F567 not transcribed to phoneme names
- **Complete subroutine catalog** — partial index in Appendix I; full \$E000-FFFF map not yet mined

Previous: Chapter 4 · **Next:** Chapter 6 — I/O guide

Chapter 6 — Input/Output Guide

This chapter describes memory-mapped hardware, the remote keypad, cartridge slot, and the 27 MHz radio control link.

Memory map summary

Region	Address	Purpose
Zero page	\$0000–\$00FF	Flags, pointers, keypad state
Warm start	\$0100–\$0103	Copyright mirror
Stack	\$01FF ↓	6502 stack
Program RAM	\$0200–\$03FE	Bytecode (2 bytes/step)
Music RAM	\$0400–\$04FF	Note pairs
Speech RAM	\$0500+	Custom phrases
Timer / misc	\$1000	I/O
Display	\$1200	Shift-register LEDs
Speech chip	\$1400	Parallel phoneme output
Motors	\$1600, \$1C00	Drive and joint timing
Cartridge ROM	\$2000–\$B000	4 KB slot stepping
Internal ROM	\$E000–\$FFFF	8 KB operating system

Full table: Appendix C.

Hardware pin map and IC inventory: Mainboard/Schematic/MMIO-Pin-Map.md, IC-Inventory.md.
ROM access listing: MMIO-ROM-Crossref.md (generated by tools/gen_mmio_crossref.py).

Memory-mapped I/O (MMIO)

The 6502 has no dedicated I/O ports; peripherals are decoded in the **\$1000–\$1FFF** page (**A12 = 1**). Sub-addresses use **A11**, **A10**, and **A9** (see decode table in MMIO-Pin-Map.md).

Address decode summary

Address	A11	A10	A9	Peripheral	Confidence
\$1000	0	0	0	Timer / display handshake	provisional
\$1200	0	0	1	LED shift register	provisional
\$1400	0	1	0	Speech parallel port	provisional
\$1600	0	1	1	Motor aux / music attack	provisional
\$1C00	1	1	0	Music tone period	provisional
\$1E00	1	1	1	System power latch	provisional

\$1000 — timer and display handshake

Bit	ROM evidence	Meaning
7	BIT \$1000 after STA \$1200 (\$ED5F–\$EDA0)	Busy — ROM waits until bit 7 clears before next display bit
0–6	STA \$1000 in IRQ (\$EE2)	Timer/divider latch (details TBD)

Driver sites: \$E08D (\$EEEE init), \$ED91 (LDA \$1000), \$EE2 (STA \$1000).

\$1200 — display

Access	ROM routine	Behavior
STA	\$ED4F, \$ED55, \$ED7B	Shift one bit into the LED chain
STX #\$00	\$E047, \$E080	Boot-time display clear / init
BIT (IRQ 7)	\$F483	Tests bit 7 during speech IRQ phase

Display wait loops are bypassed in `tools/maxxbas/patches.json` for simulation.

Hardware: COP41xL subsystem (refdes **U500**, provisional) — `National-COP41xL-Display-Motors.pdf`.

MMIO \$1400 — speech

Bit / field	ROM evidence	Meaning
D3-D0	STA \$1400 with AND #\$0F	Phoneme nybble to synthesizer
7	ORA #\$80 then write	Strobe — toggle bit 7 high then low (\$F3F9-\$F401)
4	AND #\$EF path (\$F46C)	Clock LO nybble after HO
7 (read)	BIT \$1400 in game (\$FC96)	Chip status (BPL branch)

CPU port names (parallel path): **C0-C4 Speech**, **SPBusyB** → ZP \$5B, **SPStartB** — `Maxx-Steele-CPU-Pinout.pdf`.

ROM routines: \$F3D5/\$F3D8, \$F460, \$F465. Simulator forces speech ready at \$F438 (`patches.json`).

\$1600 — motor auxiliary / music attack

Access	ROM evidence	Meaning
STA	\$F21C, \$F14B-\$F14D	Music voice attack; clear on speech end
BIT (V flag)	\$EE36	Reveille path when bit 6 set

\$1C00 — music tone / motor timing

Access	ROM evidence	Meaning
STA (4× sequence)	\$F21C	Period register for square-wave voice; OR mask from \$F2C0,X
—	\$F207	Music IRQ output target

Also drives **AudioOut Music** (CPU pin 10) timing indirectly.

\$1E00 — system power

Value	ROM site	Effect
\$00	\$E044, \$E017	Init / NMI-related clear
\$02	\$E07B	Power-down — tight loop at \$EE83

Motion COP (serial, not MMIO)

Motion opcodes use **\$EF2E** / **\$EF40**, which set ZP **\$2B bit 6** and jump to **\$F222** (motor serial routine — listing gap). **\$EF63** waits for **\$2B = 0** (talkback).

CPU pins (provisional): **COPData**, **MoCOP Data Out**, **MoCOP Done**, **COPAckB**, **MoDataClk** — National-COP420.pdf.

Remote keypad (RF input)

The handheld transmitter uses a **COP411L** MCU scanning a row/column matrix. Key events are encoded as **27 MHz OOK** packets received by the robot superhet strip.

Matrix layout

	L7	L6	L5	L4	PK
D0	A	B	C	D	
D1	E	F	G	H	
L0	I	J	K	L	
L1	M	N	O	P	
L2	Q	R	S	T	
L3	U	V	W	X	
Gnd					Y



Faceplate labels (from

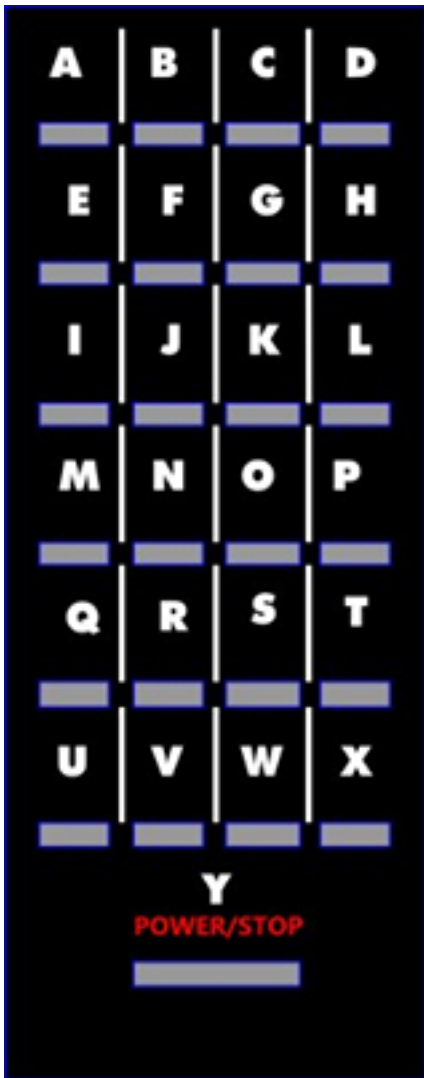
): U/1/2/3 DRIVE,

4/5 WRIST, 6/7 ARMS, 8/9 CLAW, A LAMP, B HOME, NOTE REST (WAIT), SHIFT OCTAVE, CLEAR, ENTER, SONG/NOTES, CLOCK/STATUS, SPEECH, MOTION, GAME, PROGRAM, LEARN, EXECUTE, POWER/STOP (Y).

Full key-to-matrix table: Transmitter/remote-keypad.md. Diagram: Remote-Front.svg.

	Keyboard Matrix Addresses				
Columns	L7	L6	L5	L4	P
Rows					
D0	A	B	C	D	
D1	E	F	G	H	
L0	I	J	K	L	
L1	M	N	O	P	
L2	Q	R	S	T	
L3	U	V	W	X	
Gnd					Y

Matrix wiring refs:



RF link (summary)

- Carrier: **27 MHz** on-off keying
- MCU clock: **455 kHz** ceramic resonator (IF reference)
- Bit period: **~1.55 ms**; packet repeat **~29 ms** (Power/Stop **~21 ms**)

Details: [Transmitter/transmitter-architecture.md](#), [tools/rfcap/](#).

Demo: GNU Radio flowgraph capturing OOK RF data — screen recording of the Remote-Spectrum flowgraph decoding the 27 MHz link.

Property	Value
----------	-------

Cartridge expansion

Property	Value
ROM size	4 KB per image
Mapping	\$2000, \$6000, \$A000, ... (4 KB steps)
Demo cart base	\$A000
Required header	17-byte CBS Toys copyright at offset +2

Hardware EPROM: see DataSheets/Mitsubishi-KM2365.pdf.

On-board speech and face hardware

Speech synthesis ICs documented in DataSheets/ include Sanyo LC3100/LC8100 and Eurotechnique ET9420 family parts. The CPU writes phoneme nybbles to **\$1400** (bitfield table in **MMIO \$1400 — speech** above).

Face/display COP41xL subsystem: National-COP41xL-Display-Motors.pdf (refdes **U500**, provisional).

Full IC list with confidence tiers: Mainboard/Schematic/IC-Inventory.md.

CPU and support chips

Refdes	Part	Role	Datasheet
—	MOS 6502	Main CPU	MOS-6502.pdf
U401	M6116	2K static RAM	Mitsubishi-M6116.pdf
U500	COP41xL	Display / motors (provisional)	National-COP41xL-Display-Motors.pdf
TBD	LC8100 / ET9420	Speech (variant TBD)	Sanyo-LC8100.pdf, Eurotechnique-ET9420.pdf
TBD	COP420	Motion serial COP	National-COP420.pdf

CPU pinout notes (project doc, known errors): Maxx-Steele-CPU-Pinout.pdf. Mapped to MMIO in MMIO-Pin-Map.md.

Schematics

See Schematic diagram index for board-level drawings.

Previous: Chapter 5 · **Next:** Chapter 7 — UltraMaxx BASIC

Chapter 7 — UltraMaxx BASIC Language

This chapter describes **UltraMaxx BASIC** — a line-oriented programming language for authoring Maxx Steele cartridge programs without hand-editing bytecode. UltraMaxx BASIC is a **compiler language**, not an interpreter: each statement becomes one (**opcode**, **operand**) pair in the cartridge program table (see Chapter 1).

UltraMaxx BASIC is intentionally small. It is **not** Commodore 64 BASIC: there are no variables, no IF/THEN, no GOTO, and no arithmetic expressions. Think of it as a readable front end to the opcode vocabulary in Chapter 2.

Tools: tools/maxx (command line), tools/maxxbas/ (Rust compiler).

Example sources: ultramaxx.bas, cbsdemo.bas, hello.bas.

Introduction

The factory demo cartridge (CBSDemo.532) stores its 38-step sequence as raw bytes at ROM address **\$A035**. UltraMaxx BASIC lets you write the same sequence as text:

```
FORWARD 20
DELAY 4
ARMS UP 40
END
```

The compiler emits a standard **4096-byte** cartridge image (.532) with bootstrap stub, copyright string, program table, and optional phrase/music tables — the layout described in Chapter 5.

After compile, upload the image with PicoROM or burn EPROM. The robot runs the program through the normal internal ROM executor at **\$E000**; the cartridge does not interpret BASIC at runtime.

Getting started

Add the toolchain (tools/bin/) to your path once:

```
export PATH="$(git rev-parse --show-toplevel)/tools/bin:$PATH"
```

Compile a source file:

```
maxx compile myprogram.bas -o mycart.532 --copyright ultramaxx
maxx validate mycart.532
```

Upload to a socketed PicoROM:

```
maxx upload myprogram.bas --device maxx_cart --copyright ultramaxx
```

Parse without writing output:

```
maxx check myprogram.bas
```

List program steps (JSON for simulators):

```
maxx list mycart.532 --json
```

Program format

Lines

A program is a text file, one statement per line. Blank lines are ignored.

Line numbers are optional and may be used for editor convenience (as in C64 listings). The compiler strips a leading numeric prefix:

```
10 DELAY 2
20 FORWARD 20
30 END
```


Comments

Two comment forms are accepted:

Form	Example
REM at start of line	REM wait for motors
REM after a statement	FORWARD 20 REM short roll
# to end of line	LAMP ON # headlamp

Comments are not stored in the ROM image.

Statement syntax

- Keywords are **case-insensitive** (delay, DELAY, Delay are equivalent).
- Operands are unsigned integers **0-255** (decimal or 0x hex).
- Multi-word keywords use spaces: LAMP ON, ARMS UP, CLAW OPEN.

Program terminator

Every program must end with **END**, which compiles to **FF FF**. If you omit END, the compiler appends it automatically.

Program size limits

Limit	Value	Notes
Steps per cartridge program	38 pairs max	Program ROM area \$A035-\$A080 (76 bytes)
Operand range	0-255	One byte per step
Factory demo length	38 pairs	Fills the cartridge program slot
RAM program buffer	~255 pairs	\$0200 region; larger than cart ROM

If you exceed 38 pairs, the compiler reports **program too large**. Short programs (for example hello.bas) are valid; unused program bytes in ROM are filled with **\$FF**.

The robot may accept longer sequences entered from the remote in Program mode, but a **standard 4 KB cart image** cannot ship more than 38 compiled steps without changing the fixed table layout.

UltraMaxx BASIC statements

Each statement maps to one bytecode pair. The **Byte** column shows the emitted (opcode, operand) in hex.

DELAY

Format: DELAY *seconds*

Action: Pause execution. Operand is delay time in **seconds** (opcode \$0C).

Example	Bytes
DELAY 2	0C 02
DELAY 10	0C 0A

Drive: FORWARD, BACK, LEFT, RIGHT

Format: FORWARD n · BACK n · LEFT n · RIGHT n

Action: Drive or turn. Operand scaling is empirical — see Chapter 3.

Statement	Opcode	Demo example
FORWARD 20	\$01	01 14
BACK 20	\$02	02 14
LEFT 5	\$00	00 05
RIGHT 6	\$03	03 06

ARMS UP / ARMS DOWN

Format: ARMS UP n · ARMS DOWN n

Action: Raise or lower both arms (opcodes \$06 / \$07).

Example	Bytes
ARMS UP 40	06 28
ARMS DOWN 30	07 1E

WRIST UP / WRIST DOWN

Format: WRIST UP n · WRIST DOWN n

Action: Wrist joint motion (opcodes \$04 / \$05).

Example	Bytes
WRIST DOWN 35	05 23

CLAW ROTATE / CLAW OPEN / CLAW CLOSE

Format: CLAW ROTATE n · CLAW OPEN · CLAW CLOSE

Action: Claw rotate (\$08), open (\$09/\$00), close (\$09/\$01).

Example	Bytes
CLAW ROTATE 21	08 15
CLAW OPEN	09 00
CLAW CLOSE	09 01

LAMP ON / LAMP OFF

Format: LAMP ON · LAMP OFF

Action: Head lamp (opcode \$0A).

Example	Bytes
LAMP ON	0A 01
LAMP OFF	0A 00

HOME

Format: HOME

Action: Return all joints to reference position. Operand is always **\$00**.

Example	Bytes
HOME	0B 00

PLAY

Format: PLAY *tune*

Action: Play a tune from the cartridge music table (opcode \$81). Tune data lives at ROM **\$A0BB** (copied to RAM **\$0400**). See Chapter 4.

Example	Bytes
PLAY 6	81 06
PLAY 0	81 00

SAY

Format: SAY *phrase*

Action: Speak a **RAM phrase** by slot number (opcode \$83). Phrase bytes must exist in the cartridge phrase table at **\$A081** (copied to RAM **\$0500**).

Example	Bytes	Factory demo text
SAY 16	83 10	"Hello, I am Maxx Steele"
SAY 0	83 00	"I am great, and you"

UltraMaxx BASIC does **not** yet compile quoted strings (SAY "HELLO") into phoneme nybbles. Use **SPEAK** for built-in ROM speech, or compile with **--tables-from** to copy phrase bytes from a reference ROM (see \$Phrase and music tables below).

SPEAK

Format: SPEAK *phrase*

Action: Speak a **ROM phrase** by index (opcode \$82). Uses built-in phoneme data in internal ROM — no cart phrase table required.

Example	Bytes	Notes
SPEAK 63	82 3F	Laugh (factory demo)

END

Format: END

Action: Terminate the program. Both bytes must be **\$FF**.

Example	Bytes
END	FF FF

Statement summary table

UltraMaxx BASIC	Opcode	Operand
DELAY <i>n</i>	\$0C	seconds
FORWARD <i>n</i>	\$01	distance
BACK <i>n</i>	\$02	distance
LEFT <i>n</i>	\$00	distance/angle
RIGHT <i>n</i>	\$03	angle
WRIST UP <i>n</i>	\$04	value
WRIST DOWN <i>n</i>	\$05	value
ARMS UP <i>n</i>	\$06	value
ARMS DOWN <i>n</i>	\$07	value
CLAW ROTATE <i>n</i>	\$08	value
CLAW OPEN	\$09	\$00
CLAW CLOSE	\$09	\$01
LAMP ON / OFF	\$0A	\$01 / \$00
HOME	\$0B	\$00
PLAY <i>n</i>	\$81	tune index
SPEAK <i>n</i>	\$82	ROM phrase
SAY <i>n</i>	\$83	RAM phrase slot
END	\$FF	\$FF

Copyright strings

The compiler writes a 17-byte copyright field at ROM offset **\$02**. Choose with **--copyright:**

Value	String	Typical use
ultramaxx	(c) UltraMaxx	Community cartridges
cbs	(c) 1985 CBS Toys	CBS factory branding

Cartridge detection in internal ROM compares this field during warm start.

Phrase and music tables

Programs that use **SAY** or **PLAY** need matching data in the cart tables at **\$A081** and **\$A0BB**. The compiler fills these regions with **\$FF** / **\$00** unless you supply a reference image:

Example (ultramaxx.bas, tables from UltraMaxx.532):

```
maxx compile ultramaxx.bas -o UltraMaxx.bas.532 \  
  --copyright ultramaxx \  
  --tables-from Cartridge/Examples/UltraMaxx/Firmware/Binary/UltraMaxx.532
```

This copies phrase and music bytes from the reference ROM while replacing the program table with your compiled source. The full factory demo in ultramaxx.bas compiles **byte-identical** to stock UltraMaxx.532 when tables are copied from that file.

Custom speech authoring (phoneme encoding) remains future work — see Chapter 4 and Cartridge/PROGRAMMING.md §10.

Sample programs

Minimal program

```
DELAY 1  
FORWARD 20
```

```
LAMP ON
DELAY 3
LAMP OFF
HOME
END
```

Source: hello.bas.

Factory demo (38 steps)

The complete CBS/UltraMaxx demonstration sequence — speech, motion, lamp, tunes, home, and backup — is available as:

Branding	Source
CBS Toys	cbsdemo.bas
UltraMaxx	ultramaxx.bas

Both sources compile to the same motion program; only the copyright string differs.

Compiler errors

UltraMaxx BASIC reports line-numbered errors at compile time (not on the robot display).

Message	Cause
empty program	No statements in file
unknown statement	Keyword not in vocabulary (e.g. GOTO, PRINT)
operand ... out of range 0..255	Numeric operand too large
program too large	More than 38 pairs
DELAY requires one operand	Missing or extra operands
LAMP requires ON or OFF	Invalid lamp sub-keyword
validation failed	Emitted image failed cart structure checks

Use **maxx check file** to validate syntax without writing a .532 file.

What UltraMaxx BASIC is not

The following Commodore 64 BASIC features are **intentionally absent**:

C64 BASIC	UltraMaxx BASIC
Variables (A, X\$)	Not supported
IF / THEN	Not supported
GOTO / GOSUB	Not supported
FOR / NEXT	Not supported
Arithmetic expressions	Not supported
INPUT / READ / DATA	Not supported
String functions	Not supported
Inline interpreter	Not supported — compile only

Control flow is **strictly linear**: the executor runs steps in order from \$0200 until **END**.

Relationship to other chapters

Topic	See
Bytecode rules, modes, RAM limits	Chapter 1
Opcode hex reference	Chapter 2
Motion operand tuning	Chapter 3
Speech phonemes, music bytes	Chapter 4
Bootstrap stub, ROM map	Chapter 5
Cartridge authoring workflow	Cartridge/PROGRAMMING.md
PicoROM upload	Cartridge/Examples/UltraMaxx/PICOROM.m

Quick command reference

All commands are provided by `tools/maxx` (see `tools/maxxbas/` for the Rust implementation).

Command	Purpose
<code>maxx compile file.bas</code>	Build .532 image
<code>maxx check file.bas</code>	Syntax check
<code>maxx validate file.532</code>	Verify cart structure
<code>maxx list file.532</code>	Human-readable step list
<code>maxx list --json</code>	Machine-readable trace (simulators)
<code>maxx upload file</code>	Compile (if needed) + PicoROM
<code>maxx simulate file</code>	Step preview (<code>tools/maxxbas/</code> ; add <code>--gui</code> for interactive playback)

Previous: Chapter 6 — Input/output guide · **Next:** Appendices

Appendices

Reference tables for the Maxx Steele Programmer's Reference Guide.

A — Opcode abbreviations

See Quick reference for the full hex table. Names match tools/maxx_rom.py OPCODES dict.

B — Display segment table

Internal ROM table **\$F878** maps opcode indices to two-character LED segment patterns. Documented display names:

Index	Display	Opcode context
—	L, F, b, r	Drive
—	Uu, Ud, Au, Ad, Cr, Cc	Joints
—	HL, init, d	Lamp, home, delay
—	PLAY, SPEE, SS, CLr	Extended
—	End, beg	\$FF, \$FE

Full glyph bitmaps are embedded in maxx_internal_ROM.dsm near label FULL / \$F878 references.

C — Memory map

Region	Address	Size	Purpose
Zero page	\$0000–\$00FF	256 B	Flags, pointers
Warm start	\$0100–\$0103	4 B	Copyright mirror
Stack	\$01FF ↓	—	6502 stack
Program RAM	\$0200–\$03FE	510 B	Bytecode
Music RAM	\$0400–\$04FF	256 B	Note pairs
Speech RAM	\$0500+	—	Phrases for \$83
I/O	\$1000	—	Timer
Display	\$1200	—	LED shift register
Speech	\$1400	—	Phoneme parallel port
Motors	\$1600, \$1C00	—	Motor drive
Cartridge	\$2000–\$B000	4K slots	Plug-in ROM
Internal ROM	\$E000–\$FFFF	8 KB	OS / interpreter

D — Zero-page registers

Status and mode

Addr	Name	Purpose
\$02	Status A	Speech error, backoff, power, drive flags (Appendix E)
\$03	Status B	User control, speech enable, execute gating
\$04	Reveille flag	Set by clock path; cleared at \$EE32
\$0D	Mode	0=immediate, 1=learn, 2=program, 3=execute, 4=game
\$0E	Entry state	Non-zero during multi-key opcode entry

Addr	Name	Purpose
\$0F/\$10	Program pointer	Address of current step in \$0200 (beg = \$0180)
\$11/\$13 \$15	Current op/operand Keycode	Fetches by \$E519 for executor Last decoded keypad value from \$E617
\$24/\$25	Executor pointer	Working address; displayed for extended opcodes
\$2B	Talkback busy	Non-zero while motors/speech pending; \$EF63 waits
\$26 \$27	Timer prescale Second timer	IRQ countdown; reset to \$F4 at \$E99 Game delays, enter-key timeout (Moon Ball ready ≈ \$14 s; replay wait \$10 s)
\$A6	Game difficulty / points	Moon Ball: level 0-3 and BCD points per return; Force Field: level × 4
\$A7	Game score A	Moon Ball: player BCD low; Force Field: player score → display \$24
\$A8	Game score B	Moon Ball: player BCD high; Force Field: Maxx score → display \$25
\$A9	Game counter	Moon Ball: rounds until difficulty ramp; Force Field: level phase count
\$AA	Game sub-timer	Force Field: shield-down / laser-burst phases
\$AB	Game keypad flags	Force Field: shield/laser state from \$FA0C + mask \$FA98
\$AC	Miss counter	Moon Ball: starts at 3
\$AD	Game scratch	Cleared by \$F9DE; purpose not fully traced
\$39-\$3C	Music state	Pointers/flags during note entry (\$EF76 region)
\$56	IRQ phase	Selects handler in IRQ jump table
\$57	Speech timing	Used during \$1400 nybble output
\$5B	Speech status	Error/ready flags for LC8100 path
\$67-\$6A	Motor timing	Decrement in IRQ; init to \$FF at warm start
\$72-\$96	Vector table	Copied from ROM \$E01C at boot (see below)
\$74	Tune index	Last PLAY operand; power-down tune selection
\$75	Keypad buffer	Bit 7 = key ready; \$E600 waits for match
\$8E/\$8F	Cart pointer	16-bit cartridge scan address (page in \$8F)
\$96	Energy counter	Decrement in IRQ; triggers low-battery speech

Vector table (\$72-\$96, from ROM \$E01C)

ZP	Initial target	Role
\$72/\$73	\$0082	Reserved / table header
\$76/\$77	\$E014	NMI vector
\$78/\$79	\$FDC8	IRQ vector
\$7A/\$7B	\$E0CB	Return to main loop (immediate)
\$7C-\$85	\$F1D8...	Music IRQ dispatch table
\$8C/\$8D	\$EE32	Opcode dispatch (\$EE2F)
\$8E/\$8F	\$1000	Cartridge entry pointer
\$90/\$91	\$E598	Alternate handler vector
\$92/\$93	\$F3D8	Speech output vector
\$94/\$95	\$F8CE	Game mode entry

E — Status bits

\$02 (from tools/maxx_rom.py)

Bit	Label
0	SER (speech error)
1	Ebof (enable backoff)
2	PDOn (power down on)
3	Edof (enable drive off)
4	SPon (speech on)

Keypad toggles (internal ROM): row 0 → SER, PAR; row 2 → Edon, Edof; row 3 → UCon, UCoF; row 4 → Ebon, Ebof; row 5 → SPon, SPof.

\$03

Bit	Label
0	UCon (user control on)
1	SPon (speech enable)

Demo bootstrap: \$02 ← \$02, \$03 ← \$82.

F — Music duration table

Note durations for the music IRQ path are read from **\$F15B** (indexed 1-8 in maxx_internal_ROM.dsm). Frequency/note bytes live in **\$0400**.

Complete note name → byte mapping is not yet transcribed in this archive.

G — Cartridge file layout

File offset = cartridge address − \$A000 for demo cart base.

File offset	Addr	Content
\$0000	\$A000	Entry vector
\$0002	\$A002	Copyright (17 bytes)
\$0013	\$A013	Bootstrap code
\$0035	\$A035	Program table
\$0081	\$A081	Phrase table
\$00BB	\$A0BB	Music table

File offset	Addr	Content
-------------	------	---------

H — tools/maxx_rom.py commands

```
python3 tools/maxx_rom.py disasm PATH [--compare-dsm DSM]
python3 tools/maxx_rom.py validate PATH
python3 tools/maxx_rom.py template OUTPUT.532
python3 tools/maxx_rom.py opcodes OUTPUT.json
```

I — Internal ROM entry points

Curated catalog from maxx_internal_ROM.dsm. See Chapter 5 for boot flow and mode diagrams.

Boot and initialization

Address	Role
\$E041	RESET entry
\$E05A	Cold start — zero page, clear RAM tables
\$E06F	Warm start — copy vectors \$E01C → \$72
\$E08E–\$E0B3	Cartridge scan and JMP (\$008E)
\$E574	Init learn pointer, \$0200 ← \$FF
\$E582	Init program pointers (cold start)
\$E57B	Set \$0180 ← \$FF terminator
\$EEEE	Timer byte init

Main loop and modes

Address	Role
\$E0B6	Main loop entry (post-cart or no cart)
\$E0D1	Mode change — STA \$0D, re-dispatch
\$E0ED	Mode dispatch on \$0D
\$E154	Immediate mode entry
\$E161	Learn/immediate keypad loop
\$E17B	Immediate key decode (key in A)
\$E346	Program mode loop
\$E434	Execute mode runner
\$E905	Status housekeeping (called each loop)
\$F8CE	Game mode entry (JMP (\$0094))
\$F8E9	Moon Ball (game 1) main
\$F9DE	Zero \$A6–\$AD
\$F9E8	Difficulty picker (phrase \$11, 0–3)
\$F9C6	Moon Ball game over
\$FA01	Copy \$A7/\$A8 → \$24/\$25 for display
\$FA0C	Game keypad read (Force Field shield/laser)
\$FA67	Replay prompt (phrase \$1C) + ENTER wait
\$FAA4	Force Field (game 2) main
\$FB8C	Force Field shield/laser IRQ helper
\$FBCE	Force Field intermission between levels
\$FC12	Force Field score adjust (hit / ricochet penalty)
\$FC30	Force Field win check (CMP #\$25)

Address	Role
\$FC6F	Game IRQ slice (called from \$FDCD)

Keypad and program I/O

Address	Role
\$E3A9	CLEAR program function
\$E3EC	Delay / wait for flag change
\$E409	RUN preview (ENTER in program mode)
\$E41A	Increment program pointer (+2)
\$E519	Fetch opcode/operand to \$11/\$13
\$E555	Decrement pointer (BEGIN)
\$E600	Wait until \$75 matches A
\$E617	Poll keypad → \$15
\$E6A4	Key poll wrapper (motors, RF)
\$E6B5	Keycode → opcode translation table
\$E9FE	Keypad input helper (games, music entry)

Bytecode executor

Address	Role
\$EE2F	JMP (\$008C) — opcode dispatch
\$EE32	Dispatch entry (Reveille, execute paths)
\$F64E	Program step number → display
\$F66C	Execute one program command
\$F737	Compute step number from \$0F/\$10

Display

Address	Role
\$ED48	Display prefix
\$ED4F	Shift LO bit to \$1200
\$ED7B	Send byte to display
\$EDAF	Stalled-motor check
\$F684	Send 2-char string to display
\$F878	Opcode → segment pattern table

Speech

Address	Role
\$F3D5	Speech output entry (JMP (\$0092))
\$F3D8	Phoneme output (index X)
\$F40F	Say ROM phrase by index (X)
\$F460 / \$F465	Clock nybbles to \$1400
\$F475	Say built-in phrase (index X)

Address	Role
\$F4DB	Phoneme duration table (140 entries)
\$F567	Phoneme code table

Music

Address	Role
\$EF01	Resolve tune pointer for operand
\$EF55	Clear \$0400 music table
\$EF76	Leave learn / music cleanup
\$F0B8	Append note to \$0400
\$F15B	Note duration table
\$F1D8	Music IRQ handler region

Motors

Address	Role
\$EF2E	Motor drive command
\$EF40	Stop motors
\$EF63	Wait for talkback (\$2B = 0)

J — Built-in speech phrases (game subset)

ROM table at **\$F640**; say via **\$F40F** / **\$F475**. Factory User Manual numbers phrases **16-32** (decimal); ROM uses **\$10-\$20** (hex).

Manual #	ROM (hex)	Phrase	Game use
16	\$10	Hello. I am Maxx Steele.	Power-on greeting
17	\$12	Please choose game.	Game mode entry
18	\$11	Please choose how tough.	After game selected
19	\$13	Good play.	Moon Ball successful return
24	\$18	Maxx Steele wins.	Force Field — Maxx wins
25	\$19	Congratulations, you win.	Force Field — player wins
27	\$1B	Game over.	Moon Ball end
28	\$1C	Choose enter to play again.	Both games replay prompt
32	\$20	I'm ready.	Moon Ball ready phase

Player rules: User Manual Ch 6. ROM mapping: Chapter 5 — Game mode.

K — Messages and errors

Message / flag	Cause
FULL	Program exceeds \$037F / RAM limit
Ser	Speech error status bit
validate copyright mismatch	Cart missing (c) 1985 CBS Toys
Missing FF FF	Program table not terminated

L — Bibliography

- R. Wind — `maxx_internal_ROM.dsm`, `maxx_demo_ROM_532.dsm` (2002–2006)
- Factory manual — `Chassis/Manual/MaxxSteeleReferenceGuide.pdf`
- This repository — <https://github.com/webaugur/Maxx-Steele-1984-Robot>
- C64 Programmer’s Reference Guide — structural inspiration (Commodore, 1982)

M — MMIO hardware cross-reference

Partial pin map and IC inventory (provisional decode). Full detail: `Mainboard/Schematic/MMIO-Pin-Map.md`.

MMIO	Peripheral	Key ROM drivers	Likely IC
\$1000	Timer / display handshake	\$EEEE, \$ED5F–\$EDA0	Counter TBD
\$1200	LED shift register	\$ED4F, \$ED7B, \$F684	COP41xL U500
\$1400	Speech nybbles + strobe	\$F3D8, \$F460	LC8100 / ET9420
\$1600	Motor aux / music attack	\$F21C, \$F14B	COP41xL / COP420
\$1C00	Music tone period	\$F207, \$F21C	Audio path
\$1E00	Power latch	\$E041, \$E07B	Glue logic

ROM access listing: `MMIO-ROM-Crossref.md` (`tools/gen_mmio_crossref.py`).

N — Glossary

Term	Meaning
Bytecode	Opcode/operand pairs interpreted by internal ROM
Bootstrap	Short 6502 stub in cartridge that loads RAM tables
OOK	On-off keying RF modulation (27 MHz)
IF	Intermediate frequency (455 kHz in transmitter/receiver)
Phoneme	Speech sound unit; 4-bit codes to \$1400
Refdes	Schematic reference designator (U1, U400, ...)
.532	4 KB cartridge image file extension used in archive

Maxx Steele Quick Reference Card

Single-page summary. Full detail in chapters 1-7 and Appendices.

Modes (\$0D)

Val	Mode
0	Immediate
1	Learn
2	Program
3	Execute

Key zero page

Addr	Use
\$02	Status A
\$03	Status B
\$0D	Mode
\$0F/\$10	Program step pointer
\$0200	Program RAM base
\$0400	Music RAM
\$0500	Speech RAM

Motion opcodes

Hex	Disp	Action
00	L	Left
01	F	Forward
02	b	Reverse
03	r	Right
04	Uu	Wrist up
05	Ud	Wrist down
06	Au	Arms up
07	Ad	Arms down
08	Cr	Claw rotate
09	Cc	Claw open/close
0A	HL	Lamp
0B	init	Home (00)
0C	d	Delay (sec)

Speech / music

Hex	Disp	Action
0E	S	Speech ROM
0F	SS	Speech shift
10	PS	Program speech
81	PLAY	Play tune

Hex	Disp	Action
82	SPEE	Speak ROM
83	SS	Speak RAM
84	CLr	Clear phrase
FE	beg	Marker
FF	End	End (FF FF)

Cartridge header (\$A000)

Off	Content
+0	Entry vector
+2	(c) 1985 CBS Toys
+\$13	Bootstrap
+\$35	Program
+81 <i>Phrases</i> +BB	Music

Common sequences

0C nn delay nn seconds
 0B 00 home
 0A 01 lamp on
 0A 00 lamp off
 FF FF end program

UltraMaxx BASIC (Ch 7)

Statement	Example
Delay	DELAY 2
Drive	FORWARD 20 BACK 20 LEFT 5 RIGHT 6
Joints	ARMS UP 40 WRIST DOWN 35 CLAW ROTATE 21 CLAW OPEN
Lamp	LAMP ON / LAMP OFF
Home	HOME
Music	PLAY 6
Speech	SPEAK 63 (ROM) · SAY 0 (RAM slot)
End	END

Cart limit: 38 steps max. **Not supported:** variables, IF, GOTO.

CLI (tools/maxx):

```
maxx compile FILE.bas -o FILE.532
maxx upload FILE.bas --device maxx_cart
```

Tools

Binaries: tools/maxx, tools/maxx_rom.py (shell wrapper tools/bin/maxx-rom).

```
maxx compile FILE.bas -o FILE.532
maxx-rom validate FILE.532
maxx-rom template FILE.532
```

Schematic Diagram Index

The Commodore 64 Programmer's Reference included a fold-out schematic. This index points to equivalent materials in the Maxx Steele archive.

Main logic board

Asset	Path	Status
Raster schematic (enhanced) SVG (v1.1)	Mainboard/Schematic/MaxxSteele/Schematic_enh-v1.1.png	Available
KiCad project	Mainboard/Schematic/v1.1/MaxxSteele_Schematic.svg	Available
	Mainboard/KiCAD/	Planned — digitization in progress

6502 CPU, RAM (M6116), cartridge interface, speech and motor glue.

MMIO hardware (partial): Mainboard/Schematic/MMIO-Pin-Map.md, IC-Inventory.md.

Transmitter (remote)

Asset	Path	Status
KiCad schematic + PCB	Transmitter/KiCAD/Transmitter-27MHz.kicad_pro	Active
RE notes	Transmitter/ReverseEngineering	Available
BOM	Transmitter/transmitter-Available bom.md	Available

COP411L (U1), 455 kHz clock, 27 MHz OOK stage.

Receiver (robot RF)

Asset	Path	Status
KiCad schematic	Receiver/KiCAD/Receiver-Schematic only 27MHz.kicad_pro	Available
PCB layout	—	Not yet in archive

Cartridge

Asset	Path	Status
KiCad (CBSDemo)	Cartridge/Examples/CBSDemo/KiCAD/CBSDemo.kicad_pro	Active
KiCad (UltraMaxx)	Cartridge/Examples/UltraMaxx/KiCAD/ (same PCB as CBSDemo)	Active
PicoROM (UltraMaxx U1)	Cartridge/Examples/UltraMaxx/PICOROM.md	Active

Face / speech / display

Asset	Path	Status
KiCad Datasheets	Face/KiCAD/ DataSheets/	Planned LC3100, LC8100, ET9420, COP41xL

Power module

Asset	Path	Status
KiCad	Power/KiCAD/	Planned

Paddle mirror accessory

Asset	Path	Status
Photo Schematic	PaddleMirror/Photos/ —	Available Needed

Chip cross-reference

Indexed datasheets with refdes where known: DataSheets/README.md.

ABOUT THE MAXX STEELE... ROBO FORCE TECHNICAL MANUAL...

Action figure compatibility... Advanced Robotic Speech and Sound... and high tech battle capabilities make the Maxx Steele Robot Force the most advanced robotic toy in its class for home, play, and educational use.

The MAXX STEELE ROBO FORCE TECHNICAL MANUAL tells you everything you need to know about your Maxx Steele robot. The perfect companion to your Robot Force Commander's Guide, this manual presents detailed information on everything from laser blasters to advanced machine language techniques. This book is a must for everyone from the beginner to the advanced robotic engineer.

For the beginner, the most complicated topics are explained with many sample programs and an easy-to-read wiring style. For the advanced robotic engineer, this book has been subjected to heavy pre-testing with your needs in mind. And it's designed so that you can easily get the most out of your Robot Force capabilities.
engineer.

For the beginner, the most complicated topics are explained with many sample programs and an easy-to-read wiring style. For the advanced robotic engineer, this book has been subjected to heavy pre-testing with your needs in mind. And it's designed so that you can easily get the most out of your Robot Force capabilities.



maxx steele
ROBO FORCE

Maxx Steele Robot Force, Inc. — Robot Systems Division.
1301 Western Avenue, Cincinnati, Ohio 45203